STUDENT RECORD MANAGEMENT SYSTEM

Data Structures and Algorithms (DSA) Project Report

Submitted by: Arava Abhiram sai
Roll Number: 25BAI70777
Course: BE CSE AIML
Semester: 3rd Semester
Subject: Data Structures and Algorithms
University: Chandigarh University
Academic Year: 2026–2027

---

1. Abstract

The Student Record Management System is a simple C++ application developed to manage student information using a singly linked list. It allows users to add, display, search, delete, and sort student records based on their roll numbers.

The project demonstrates important Data Structures and Algorithms concepts, including dynamic memory allocation, linked list traversal, insertion, deletion, searching, and sorting. It provides a simple menu-driven interface for managing student information.

2. Introduction

Managing student records manually can be time-consuming and may lead to errors. This project provides a simple computerized solution for storing and managing student details.

The system uses a singly linked list, in which each node stores a student's roll number, name, marks, and a pointer to the next node. The records can be modified dynamically without declaring a fixed-size array.

3. Problem Statement

Manual management of student records makes searching, updating, and deleting information difficult when the number of students increases. The objective is to develop a simple C++ application that performs these operations efficiently using a suitable data structure.

4. Objectives

- To implement a singly linked list in C++.
- To add and display student records.
- To search for a student using a roll number.
- To delete a student record.
- To sort student records by roll number.
- To understand dynamic memory allocation and pointers.

5. Software Requirements

- **Programming language:** C++
- **IDE:** Visual Studio Code
- **Compiler:** GCC/G++ (TDM-GCC)
- **Operating system:** Windows

6. Data Structure Used

Singly Linked List

A singly linked list is a linear data structure made up of nodes. Each node contains data and a pointer to the next node.

Each student node stores:

- Roll number
- Student name
- Marks
- Pointer to the next node

The head pointer stores the address of the first node. The last node points to nullptr.

7. System Modules

7.1 Add Student

Accepts a student's roll number, name, and marks, and adds a new node to the linked list. Duplicate roll numbers are rejected.

7.2 Display Students

Traverses the linked list and displays all student records.

7.3 Search Student

Searches the linked list using the roll number and displays the student's details if found.

7.4 Delete Student

Finds the requested roll number, adjusts the links, and removes the corresponding node from memory.

7.5 Sort Students

Sorts student records in ascending order of roll number using nested loops.

7.6 Exit

Ends the program and releases the dynamically allocated memory.

8. Algorithm

1. Start the program.
2. Initialize the head pointer to nullptr.
3. Display the menu of operations.
4. Accept the user's choice.
5. Perform the selected operation:
    - Add a student.
    - Display all students.
    - Search for a student.
    - Delete a student.
    - Sort students by roll number.
6. Repeat the menu until the user selects Exit.
7. Release all allocated nodes.
8. Stop the program.

9. Sample Output

===== Student Record Management System =====

1. Add Student

2. Display Students

3. Search Student

4. Delete Student

5. Sort Students by Roll Number

6. Exit

Enter your choice: 1

Enter roll number: 102

Enter student name: Rahul

Enter marks: 85

Student added successfully!

Enter your choice: 2

Roll Number: 102

Name: Rahul

Marks: 85

Enter your choice: 6

Goodbye!

10. Time Complexity Analysis

Let n be the number of student records.

Operation	Time Complexity
Add Student	O(n)
Display Students	O(n)
Search Student	O(n)
Delete Student	O(n)
Sort Students	$O(n^2)$

The add operation takes O(n) time because the program traverses the list to check for duplicate roll numbers and locate the last node.

11. Advantages

- Student records can be added dynamically.
- No fixed-size array is required.
- Searching and deleting records are straightforward.
- It provides practical experience with pointers and dynamic memory allocation.
- The menu-driven interface is easy to use.

12. Limitations

- Records are stored only in memory.
- Data is lost when the program closes.
- Searching requires traversing the list.
- The sorting algorithm is not efficient for very large datasets.

13. Future Enhancements

- Store records permanently in a file.
- Add an option to update student details.
- Search students by name.

- Calculate grades based on marks.
- Improve sorting efficiency for large datasets.

14. Conclusion

The Student Record Management System successfully demonstrates the implementation of a singly linked list in C++. It supports adding, displaying, searching, deleting, and sorting student records. The project improves understanding of pointers, dynamic memory allocation, linked list operations, and algorithmic time complexity.

15. References

1. C++ programming language documentation.
2. Data Structures and Algorithms course notes.
3. Visual Studio Code documentation.
4. GCC/G++ compiler documentation.